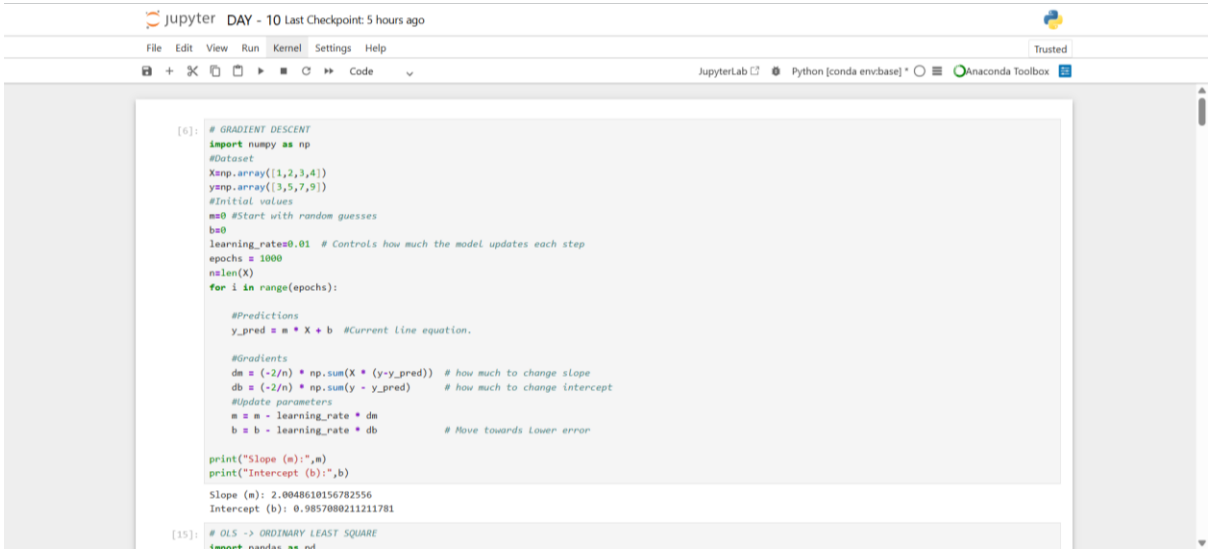
DAY-10


jupyter DAY - 10 Last Checkpoint: 5 hours ago

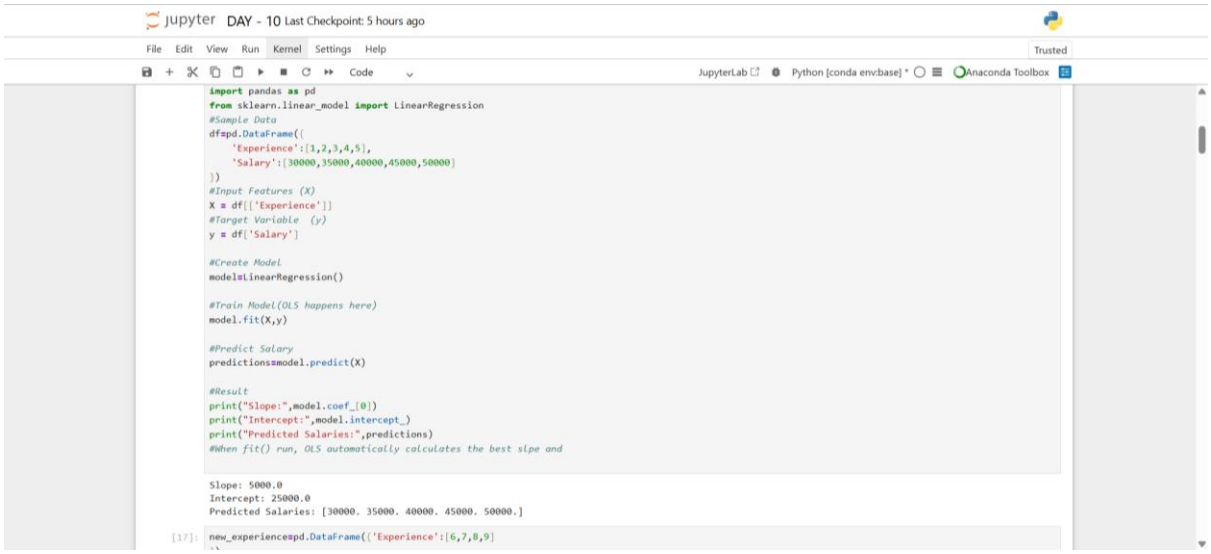
File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
[6]: # GRADIENT DESCENT
import numpy as np
#Dataset
X=np.array([1,2,3,4])
y=np.array([3,5,7,9])
#Initial values
m=0 #Start with random guesses
b=0
learning_rate=0.01  # Controls how much the model updates each step
epochs = 1000
n=len(X)
for i in range(epochs):

    #Predictions
    y_pred = m * X + b  #Current line equation.

    #Gradients
    dm = (-2/n) * np.sum(X * (y-y_pred))  # how much to change slope
    db = (-2/n) * np.sum(y - y_pred)      # how much to change intercept
    #Update parameters
    m = m - learning_rate * dm
    b = b - learning_rate * db            # Move towards lower error

print("Slope (m):",m)
print("Intercept (b):",b)
```

```
Slope (m): 2.0048610156782556
Intercept (b): 0.9857080211211781
```

```
[15]: # OLS -> ORDINARY LEAST SQUARE
import pandas as pd
```


jupyter DAY - 10 Last Checkpoint: 5 hours ago

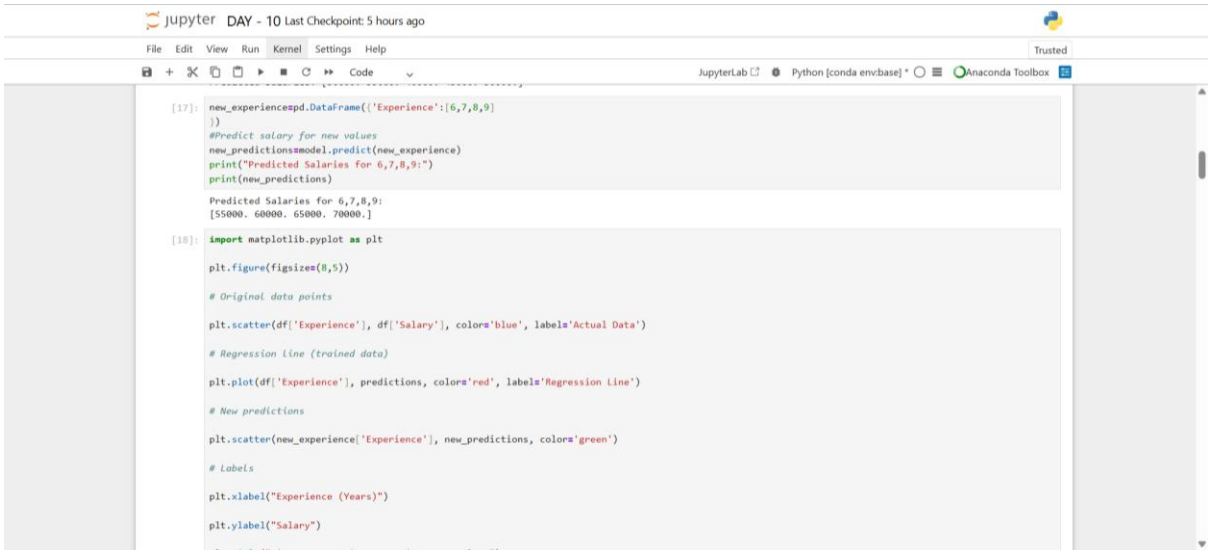
File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
import pandas as pd
from sklearn.linear_model import LinearRegression
#Sample Data
df=pd.DataFrame({
    'Experience':[1,2,3,4,5],
    'Salary':[30000,35000,40000,45000,50000]
})
#Input Features (X)
X = df[['Experience']]
#Target Variable  (y)
y = df['Salary']

#Create Model
model=LinearRegression()

#Train Model(OLS happens here)
model.fit(X,y)

#Predict Salary
predictions=model.predict(X)

#Result
print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)
print("Predicted Salaries:",predictions)
#When fit() run, OLS automatically calculates the best slpe and
```

```
Slope: 5000.0
Intercept: 25000.0
Predicted Salaries: [30000. 35000. 40000. 45000. 50000.]
```

```
[17]: new_experience=pd.DataFrame({'Experience':[6,7,8,9]
```


jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
[17]: new_experience=pd.DataFrame({'Experience':[6,7,8,9]
})
#Predict salary for new values
new_predictions=model.predict(new_experience)
print("Predicted Salaries for 6,7,8,9:")
print(new_predictions)
```

```
Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]
```

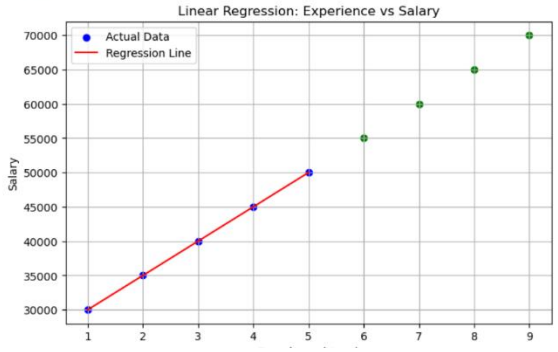
```
[18]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

# Original data points

plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual Data')

# Regression line (trained data)

plt.plot(df['Experience'], predictions, color='red', label='Regression Line')

# New predictions

plt.scatter(new_experience['Experience'], new_predictions, color='green')

# Labels

plt.xlabel("Experience (Years)")

plt.ylabel("Salary")
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
plt.title("Linear Regression: Experience vs Salary")

plt.legend()

plt.grid()

plt.show()
```



jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

[22]:
```
##Multiple Regression Code

import pandas as pd

from sklearn.linear_model import LinearRegression

# Dataset

df = pd.DataFrame({

    'Area':[1000,1200,1500,1800,2000],

    'Bedrooms':[2,3,3,4,4],

    'Age':[10,8,5,4,2],

    'Price':[300000,350000,450000,500000,600000]

})

# Features

X = df[['Area','Bedrooms','Age']]   #Using multiple features.

# Target

y = df['Price']

# Model

model = LinearRegression()
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
# Train

model.fit(X,y)   #Learns the relationship between all features and price.

# Prediction

pred = model.predict(X)   #predict house price

print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)


new=pd.DataFrame({'Area':[12000,15000,16000,17000,18000],
                  'Bedrooms':[6,7,8,9,5],
                  'Age':[11,12,13,14,15]
})
#Predict salary fro new values
new_predictions=model.predict(new)
print("Predicted Price for :")
print(new_predictions)
```

```
Coefficients: [   250.         -34615.38461538 -13461.53846154]
Intercept: 253846.15384615405
Predicted Price for :
[2898076.92307692 3600000.         3801923.07692308 4003846.15384615
 4378846.15384615]
```

[23]:
```
#VIF Code

import pandas as pd

from statsmodels.stats.outliers_influence import variance_inflation_factor
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
# Features

X = df[['Area','Bedrooms','Age']]

# VIF Calculation

vif = pd.DataFrame()

vif['Feature'] = X.columns

vif['VIF'] = [

    variance_inflation_factor(X.values, i)

    for i in range(X.shape[1])

]

print(vif)
```

```
    Feature         VIF
0      Area  165.122797
1  Bedrooms  174.483505
2       Age    2.717105
```

[29]:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
from sklearn.linear_model import LinearRegression

# Sample dataset

df = pd.DataFrame({

    'Hours': [1, 2, 3, 4, 5],

    'Marks': [10, 25, 45, 70, 100]

})

X = df[['Hours']]

y = df['Marks']

# Polynomial transformation

poly = PolynomialFeatures(degree=11)

X_poly = poly.fit_transform(X)

# Model training

model = LinearRegression()

model.fit(X_poly, y)

# Smooth curve generation (IMPORTANT STEP)

X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

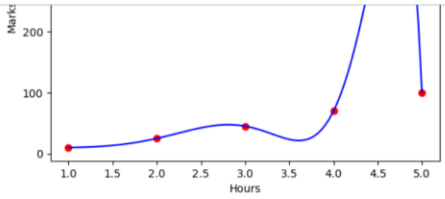
Code JupyterLab Python [conda env:base] * Anaconda Toolbox

```
X_range = pd.DataFrame(X_range, columns=['Hours'])

X_range_poly = poly.transform(X_range)

# Predictions for curve

y_curve = model.predict(X_range_poly)

# Plotting

plt.scatter(X, y, color='red', label='Actual Data')

plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')

plt.title('Polynomial Regression Curve')

plt.xlabel('Hours')

plt.ylabel('Marks')

plt.legend()

plt.show()
# WHEN DEGREE GETS CHANGED IT POLYNOMIAL CURVE ALSO CHANGES
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox



[33]:
```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)

y = np.array([2,5,8,15,28,35,50,60,85,95])

# Degree 1

poly1 = PolynomialFeatures(degree=1)

X1 = poly1.fit_transform(X)
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python [conda env:base] * Anaconda Toolbox

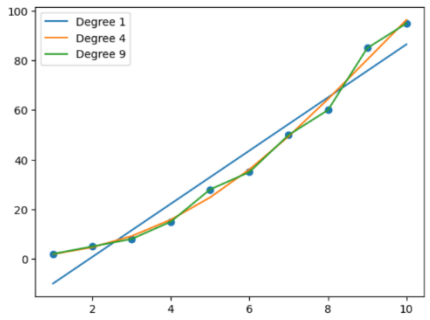
```
# Degree 4

poly4 = PolynomialFeatures(degree=4)

X4 = poly4.fit_transform(X)

# Degree 9

poly9 = PolynomialFeatures(degree=9)

X9 = poly9.fit_transform(X)

model1 = LinearRegression().fit(X1,y)

model2 = LinearRegression().fit(X4,y)

model9 = LinearRegression().fit(X9,y)

plt.scatter(X,y)

plt.plot(X, model1.predict(X1), label="Degree 1")

plt.plot(X, model2.predict(X4), label="Degree 4")

plt.plot(X, model9.predict(X9), label="Degree 9")

plt.legend()

plt.show()

#DEGREE 1 => UNDERFITTING     STRAIGHT LINE MISSES PATTERN
#DEGREE 4 => GOOD FIT         CAPTURES THE OVERALL CURVE
```

jupyter DAY - 10 Last Checkpoint: 5 hours ago

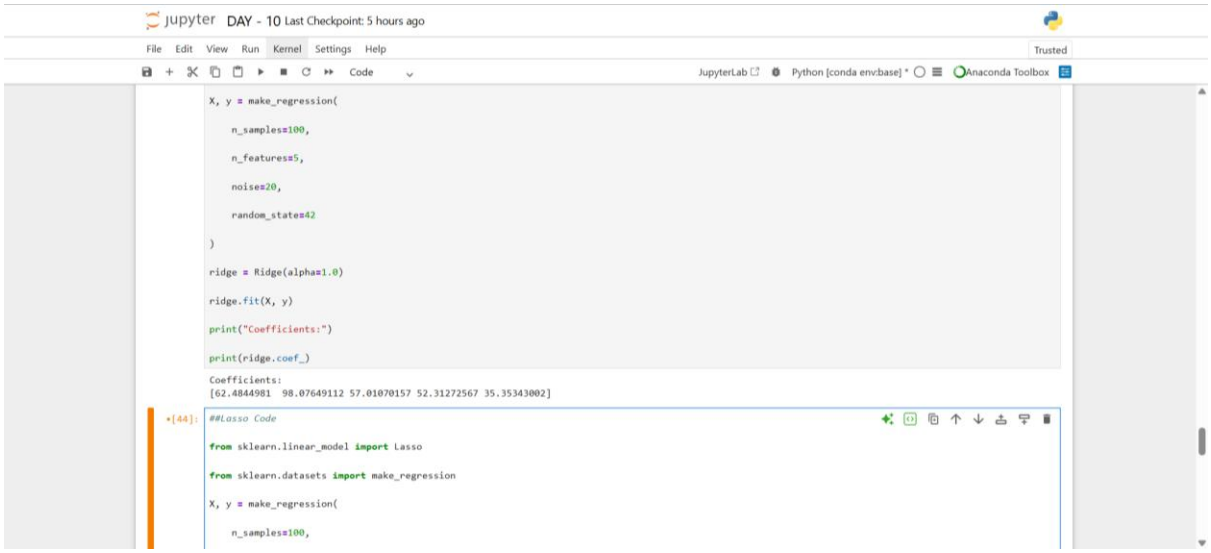
File Edit View Run Kernel Settings Help Trusted

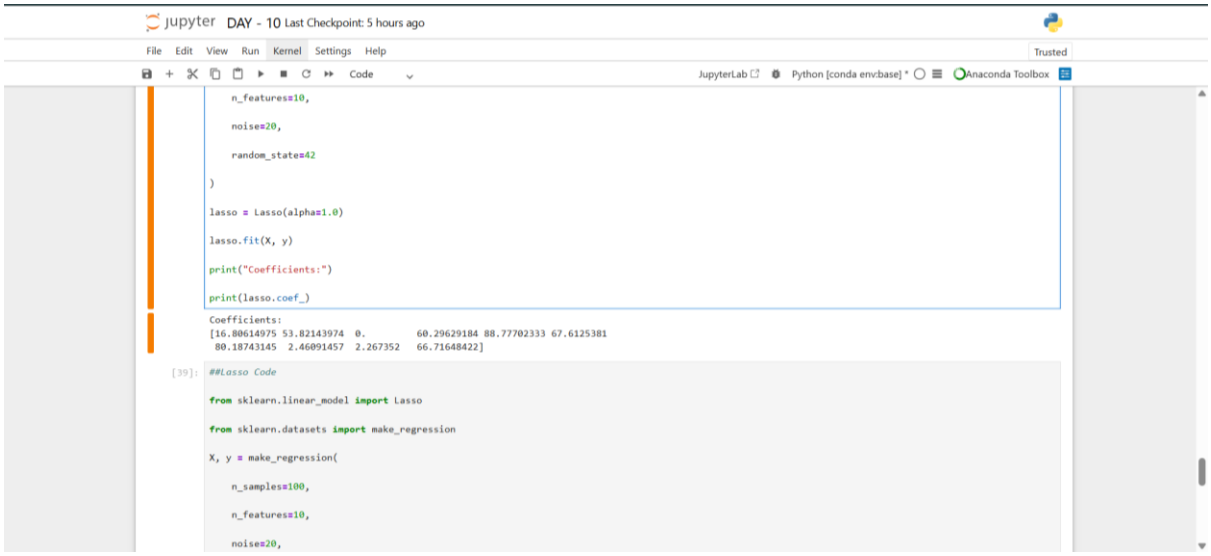
Code JupyterLab Python [conda env:base] * Anaconda Toolbox

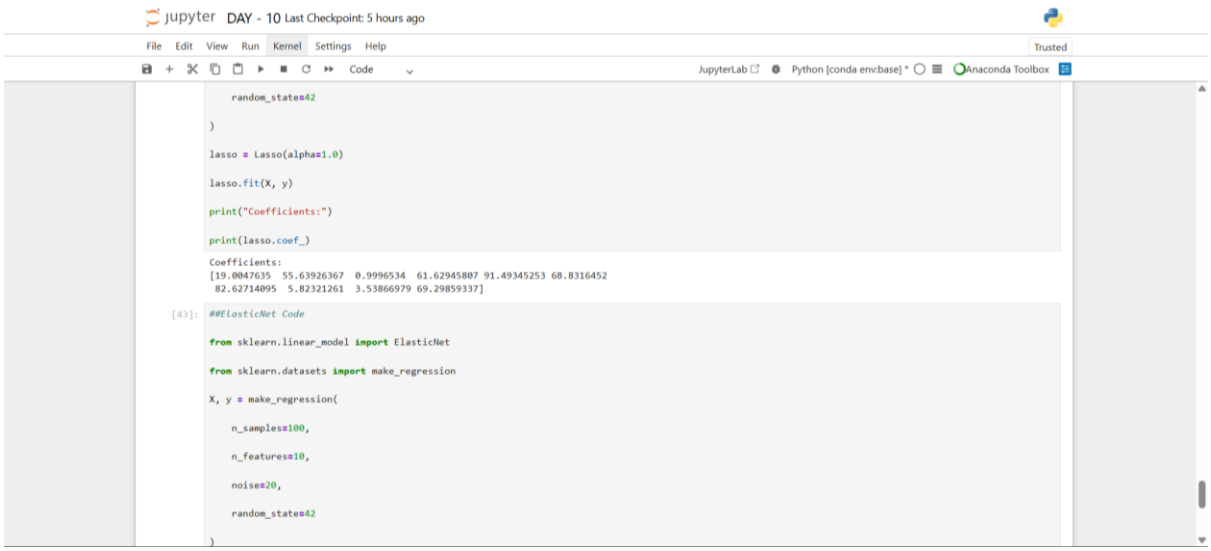
```
#DEGREE 1 => UNDERFITTING     STRAIGHT LINE MISSES PATTERN
#DEGREE 4 => GOOD FIT         CAPTURES THE OVERALL CURVE
#DEGREE 9 => OVERFITTING      TRIES TO PASS THROUGH EVERY POINT,LEARNS NOISE INSTEAD OF PATTERN
```

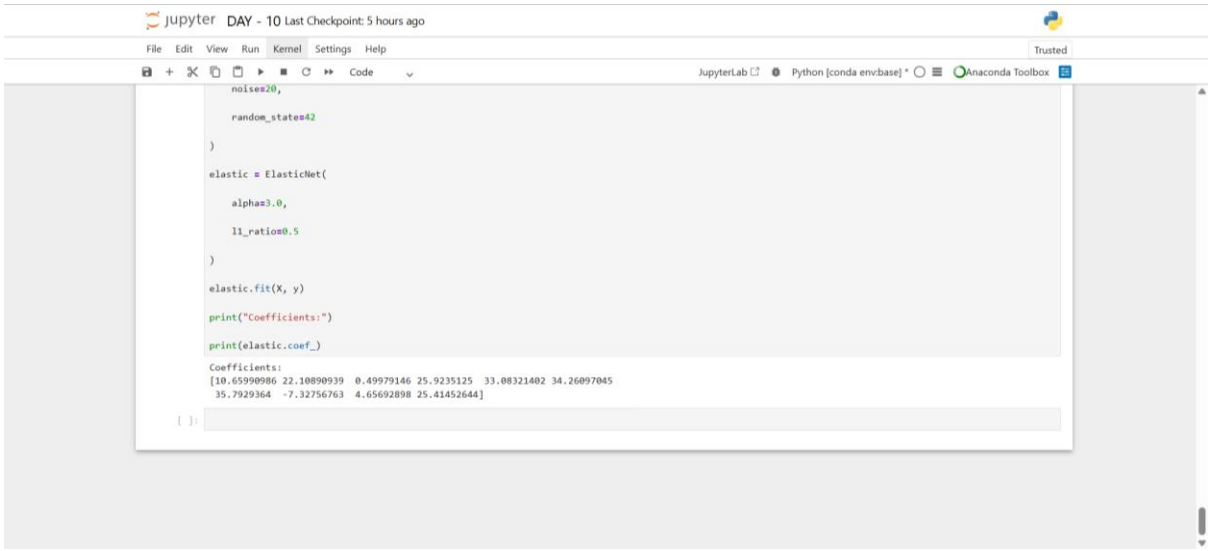


[34]:
```
##Ridge Code

from sklearn.linear_model import Ridge

from sklearn.datasets import make_regression
```


jupyter DAY - 10 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python [conda env:base] *
Anaconda Toolbox
X, y = make_regression(
n_samples=100,
n_features=5,
noise=20,
random_state=42
)
ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)
Coefficients:
[62.4844981 98.07649112 57.01070157 52.31272567 35.35343002]
*[44]:
##Lasso Code
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
n_samples=100,


jupyter DAY - 10 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python [conda env:base] *
Anaconda Toolbox
n_features=10,
noise=20,
random_state=42
)
lasso = Lasso(alpha=1.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)
Coefficients:
[16.80614975 53.82143974 0. 60.29629184 88.77702333 67.6125381
80.18743145 2.46091457 2.267352 66.71648422]
[39]:
##Lasso Code
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
n_samples=100,
n_features=10,
noise=20,


jupyter DAY - 10 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python [conda env:base] *
Anaconda Toolbox
random_state=42
)
lasso = Lasso(alpha=1.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)
Coefficients:
[19.0047635 55.63926367 0.9996534 61.62945807 91.49345253 68.8316452
82.62714095 5.82321261 3.53866979 69.29859337]
[43]:
##ElasticNet Code
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
X, y = make_regression(
n_samples=100,
n_features=10,
noise=20,
random_state=42
)


jupyter DAY - 10 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python [conda env:base] *
Anaconda Toolbox
noise=20,
random_state=42
)
elastic = ElasticNet(
alpha=3.0,
l1_ratio=0.5
)
elastic.fit(X, y)
print("Coefficients:")
print(elastic.coef_)
Coefficients:
[10.65990986 22.10890939 0.49979146 25.9235125 33.08321402 34.26097045
35.7929364 -7.32756763 4.65692898 25.41452644]
[ ]: